

FINDING

MINOR-W6-1 — HTML/EPUB chapter-title duplication: ROOT- CAUSE FINDING

Revision: 1 **Last modified:** 2026-06-16T15:11:23Z

Investigation type: BACKGROUND, READ-ONLY, NON-COMMITTING
(superpowers:systematic-debugging Iron Law — root cause FIRST; §11.4.6 no-guessing; §11.4.102). NO source edited / staged / committed. Probe binaries built to /tmp (gitignored, not staged); a temporary probe .go was created at repo root and DELETED in the same step (working tree verified clean).

1. Verdict (FACT)

The duplication is in the **PARSERS** (HTML and EPUB), NOT in bookToString and NOT in the writers. bookToString and the writers behave correctly given the data they receive — the parsers populate Section.Content with text that ALREADY contains the chapter title.

The suspected-cause hypothesis from the wave6 recorder ("bookToString concatenates Title into Content") is **PARTIALLY correct but mis-located**: bookToString DOES emit chapter.Title then section.Content (that part is true and is the SECOND copy), but the title text is duplicated *inside* section.Content by the parser BEFORE bookToString runs. So the real defect is parser-side leakage of title text into content.

2. Exact double-emission sites (file:line, proven by isolated probe)

A. HTML parser — pkg/ebook/html_parser.go

- **L47:** content := p.extractText(doc) — extractText starts from the document ROOT and in extractTextWithContext (L101-128) only skips

- `<script>/<style>`. It does **NOT skip <head> or <title>**, so the `<title>` element text is harvested into content.
- **L51: Title:** `book.Metadata.Title` — chapter Title is set to the `<title>` text (via `findTitle`, L41-44).
- Result: the `<title>` text leaks into Content, AND any `<h1>` in `<body>` is also harvested into Content. With the common pattern `<title>X</title> + <h1>X</h1>`, Content begins `"X\nX\n\n..."` — **title appears TWICE inside Content**.
- Then `cmd/unified-translator/main.go:bookToString` (L790-801) writes `chapter.Title` (a THIRD "X") then `section.Content`.

B. EPUB parser — `pkg/ebook/epub_parser.go`

- **L398-399:** the `<head>` (incl. `<title>`) IS stripped, so the `<title>` text does NOT leak (this protection exists from a prior fix — see `epub_head_multiline_bughunt_test.go`).
- **L388 + L422-428:** `chapterTitle = extractChapterTitle(content)` prefers `<title>` else `<h1>` (L448-449). The `<h1>` lives in `<body>`, which is NOT stripped — so the `<h1>` text remains in Content (L402 `removeHTMLTags` strips only the TAGS, keeps the text).
- Result: `Title="X"` AND `Content="X ..."` — **title appears ONCE inside Content**, plus the copy `bookToString` adds from `chapter.Title`.

C. `bookToString` — `cmd/unified-translator/main.go:790-801`

```
for _, chapter := range book.Chapters {
    result.WriteString(chapter.Title)    // L793 - copy #1
    result.WriteString("\n\n")
    for _, section := range chapter.Sections {
        result.WriteString(section.Content) // L796 - already
        contains the title (parser leak)
        result.WriteString("\n\n")
    }
}
```

Correct given clean input; it is NOT the defect origin.

3. Reproduction (deterministic, real system)

Built `./cmd/unified-translator` → `/tmp/ut_minor_w6` (mock provider, no API key).

Input `/tmp/minor_w6_in.html`:

```
<!DOCTYPE html><html><head><title>La Farola</title></head>
<body><h1>La Farola</h1><p>The lighthouse guided the ships.</p></
body></html>
```

Command:

```
ut -i in.html -o out.html -source-lang en -target-lang es
-provider mock -model mock Output <body> (mock translates only the first
chunk, but the dup is unmistakable):
```

```
<p>Translated: La Farola</p>
<p>La Farola La Farola</p>          <-- DUPLICATED TITLE
<p>The lighthouse guided the ships.</p>
```

Isolated HTML-parser probe (proves the leak is parser-side, pre-translate, pre-bookToString):

```
CHAPTER[0].Title="La Farola"
SECTION[0].Content="La Farola\nLa Farola\n\nThe lighthouse
guided the ships."
```

→ "La Farola" appears TWICE inside Content (title leak + h1).

Isolated EPUB-parser probe (same xhtml shape inside a real .epub):

```
CHAPTER[0].Title="La Farola"
SECTION[0].Content="La Farola The lighthouse guided the ships."
```

→ "La Farola" appears ONCE inside Content (h1 only; head stripped).

Both reproduce 100% deterministically. The matches the wave6 frame La FarolaLa Farola.

4. Precise fix recommendation

Fix at the PARSER layer (root cause), NOT in bookToString or the writers.

Two independent edits; the HTML one is primary (double leak), the EPUB one secondary (single leak).

Fix A (HTML — primary) — pkg/ebook/html_parser.go

Stop harvesting <head>/<title> into content. Cleanest, lowest-risk option: in extractTextWithContext (L101-113) skip <head> exactly as <script>/<style> are skipped:

```
if c.Type == html.ElementNode && (c.Data == "script" || c.Data ==
    "style" || c.Data == "head") {
    continue
}
```

This removes the <title>-leak copy. (It does NOT remove the <h1> body copy — see the "title-vs-h1 duplication" note below for the second-order decision.)

Fix B (EPUB — secondary) — pkg/ebook/epub_parser.go

The <head> strip already prevents the <title> leak. The residual leak is the <h1> body text. Two viable approaches (recommend the first):

1. **Do not double-render the title in bookToString** by leaving Content intact and instead NOT prepending chapter.Title when Content already begins with it — BUT that is a bookToString-side band-aid (rejected; the parser is the right layer and §11.4.6 says fix at source). Prefer: leave the <h1> in Content as the in-body heading and **stop having bookToString emit chapter.Title separately** ONLY IF the title is already the first line of Content. This is fragile.
2. **Cleaner / recommended:** treat chapter.Title as metadata only and have bookToString NOT emit it into the translatable text at all — OR have the parser NOT leave the <h1> text in Content when it was promoted to chapter.Title. Because the EPUB body legitimately needs an <h1> on round-trip (the EPUB writer re-emits <h1>%s from chapter.Title at epub_writer.go:276), the SAFEST fix is: **the EPUB writer renders the title from chapter.Title AND the parser should strip the leading <h1> from Content** so the title is carried exactly once (in chapter.Title) and re-materialised once by the writer.

The broader correct design (recommended for the committer)

Title should be carried in EXACTLY ONE place — chapter.Title (metadata) — and Section.Content should hold body text WITHOUT the title. Then:

- bookToString emitting Title + Content yields the title once (correct).
- The HTML/EPUB writers re-emit the title once from chapter.Title (correct round-trip).

Concretely: (1) HTML parser — skip <head> (Fix A) AND drop the leading <h1> that equals the title from Content; (2) EPUB parser — drop the leading <h1> that equals the title from Content (head already stripped). This makes both formats carry the title exactly once.

Avoid breaking the non-dup path

- Inputs with NO <title>/<h1> (e.g. plain .txt, HTML without head/h1): the title-strip must be a no-op — only strip an <h1>/title-line that ACTUALLY equals chapter.Title.
 - Multi-<h1> documents / real chapters: only the LEADING title occurrence (matching the promoted chapter.Title) should be removed, never interior headings.
 - DOCX/PDF/FB2 paths in generateOutput already build Book from content with titleFromInput — they are unaffected and must stay unaffected (no change there).
-

5. RED test shape (§11.4.115 — for the committer to implement)

Polarity-switch test asserting the title appears EXACTLY ONCE in the round-trip text.

Layer 1 — parser unit RED (primary, deterministic, no LLM)

pkg/ebook/html_parser_title_dup_bughunt_test.go and an EPUB sibling:

```
// RED on pre-fix code, GREEN after.
book, _ := ebook.NewHTMLParser().Parse(tmpHTML) // <title>La
           Farola</title> + <h1>La Farola</h1>
content := book.Chapters[0].Sections[0].Content
if strings.Count(content, "La Farola") != 0 { // body text must
    NOT contain the title
    t.Fatalf("title leaked into Content %d times: %q",
        strings.Count(content, "La Farola"), content)
}
// and Title carries it exactly once:
if book.Chapters[0].Title != "La Farola" { t.Fatalf(...) }
```

(EPUB variant: same assertion via UniversalParser on a built .epub fixture; mirror the existing epub_head_multiline_bughunt_test.go zip-builder helper.)

Layer 2 — bookToString / round-trip RED (cmd)

cmd/unified-translator/main_logic_test.go style — assert the flattened/extracted text has the title once:

```
got := bookToString(book) // from the title-dup HTML/EPUB fixture
if strings.Count(got, "La Farola") != 1 {
    t.Fatalf("title appears %d times in bookToString output (want
        1):\n%s",
        strings.Count(got, "La Farola"), got)
}
```

Polarity switch (§11.4.115)

Gate the assertion on RED_MODE (default 1 = reproduce defect on pre-fix artifact → asserts count>1; flip to 0 post-fix = GREEN guard asserting count==1) so the SAME source proves the defect present pre-fix and absent post-fix.

6. Evidence paths (this run)

- Repro binary: /tmp/ut_minor_w6 (gitignored, not staged)
- Repro input/output: /tmp/minor_w6_in.html, /tmp/minor_w6_out.html
- HTML-parser probe output (above, §3): Content="La Farola\nLa Farola\n\n..."

- EPUB-parser probe output (above, §3): Content="La Farola The lighthouse..."
- Wave6 frame evidence cross-ref: docs/qa/video_wave6_20260616-175612/EVIDENCE.md

7. Nothing UNCONFIRMED

All claims above are proven by isolated probes against the real parsers + a real unified-translator round-trip. No LIKELY/probably/maybe. The only design CHOICE left to the committer is which of the §4 fix variants to adopt (all converge on "title carried exactly once in chapter.Title"); that is an implementation decision, not an unproven cause.